

Debugging in Python3

Debugging Python Code with pdb

- pdb is Python's built-in command-line debugger.
- Included in the Python Standard Library.
- Helps inspect program state during execution.
- Supports breakpoints, stepping through code, and variable inspection.
- Useful for finding and fixing bugs interactively.

Introduction to pdb

- pdb stands for Python Debugger.
- Runs directly in the terminal.
- Allows execution control while the program is running.
- Requires minimal code changes.
- Supports evaluating Python expressions during debugging.

```
import pdb
```

```
pdb.set_trace()
```

```
print("Debugging started")
```

Starting a Debugger Session

- Import the pdb module.
- Call `pdb.set_trace()` where execution should pause.
- Program stops at the next executable line.
- A (Pdb) prompt appears in the terminal.
- You can inspect variables and execute commands.

```
def calculate():
```

```
    x = 10
```

```
y = 20
import pdb
pdb.set_trace()
result = x + y
return result
calculate()
```

Using breakpoint() in Python 3.7+

- breakpoint() is a shortcut for pdb.set_trace().
- Cleaner and easier to read.
- Automatically launches pdb.
- Requires no explicit import statement.
- Recommended for modern Python code.

```
def calculate():
    x = 10
    y = 20
    breakpoint()
    return x + y
```

Running a Program with pdb

- Execute the script normally using Python.
- Program pauses at the breakpoint.
- Terminal switches to pdb mode.
- Displays current file and line number.
- Awaits debugger commands.

```
python app.py
```

Basic pdb Commands

- Commands are entered at the (Pdb) prompt.

- Help is available for every command.
- Commands control execution and inspect state.
- Short aliases simplify usage.
- Most commands have one-letter versions.

Help Command

- help or h displays available commands.
- Can show documentation for specific commands.
- Useful for beginners learning pdb.
- Available during debugging sessions.
- Works interactively.

`(Pdb) help`

`(Pdb) help next`

Displaying Variables

- p prints variable values.
- Useful for inspecting program state.
- Supports Python expressions.
- Works on local and global variables.
- Commonly used while stepping through code.

`(Pdb) p name`

`(Pdb) p quantity`

Pretty Printing

- pp formats complex objects.
- Easier to read than p.
- Useful for dictionaries and nested structures.
- Displays data cleanly.
- Helps inspect API responses and JSON.

```
(Pdb) pp investment
```

Executing Python Expressions

- pdb can evaluate Python code.
- Expressions run within the current scope.
- Useful for testing logic.
- Allows temporary variable modifications.
- Helps investigate issues quickly.

```
(Pdb) p quantity * 2
```

```
(Pdb) p len(investment)
```

next Command (n)

- Executes the next line of code.
- Steps over function calls.
- Does not enter called functions.
- Useful for line-by-line debugging.
- Similar to "Step Over" in IDE debuggers.

```
(Pdb) next
```

```
(Pdb) n
```

continue Command (c)

- Resumes normal execution.
- Runs until the next breakpoint.
- Stops if another breakpoint is reached.
- Useful for skipping large sections.
- Saves time during debugging.

```
(Pdb) continue
```

```
(Pdb) c
```

quit Command (q)

- Exits the debugger immediately.
- Terminates the debugged application.
- Stops all further execution.
- Useful when debugging is complete.
- Alias: exit.

(Pdb) quit

(Pdb) q

Breakpoints

- Breakpoints pause execution at specific locations.
- Can be created dynamically during debugging.
- Help isolate problem areas.
- Support line numbers and function names.
- Essential for efficient debugging.

(Pdb) break 30

Listing Breakpoints

- Use break without arguments.
- Displays all active breakpoints.
- Shows breakpoint number and status.
- Indicates whether enabled or disabled.
- Helps manage debugging sessions.

(Pdb) break

Example Output

Num Type Disp Enb Where

1 breakpoint keep yes at app.py:30

Conditional Breakpoints

- Trigger only when a condition is true.
- Reduce unnecessary pauses.
- Useful in loops.
- Helps debug specific cases.
- Evaluates Python expressions.

```
(Pdb) break 30 quantity > 100
```

Clearing Breakpoints

- clear removes breakpoints.
- Can remove individual breakpoints.
- Can remove all breakpoints.
- Helps keep debugging organized.
- Supports breakpoint numbers.

```
(Pdb) clear
```

```
(Pdb) clear 1
```

step Command (s)

- Steps into function calls.
- Executes one statement at a time.
- Useful for tracing execution flow.
- Similar to "Step Into" in IDEs.
- Allows detailed analysis.

```
(Pdb) step
```

```
(Pdb) s
```

```
Example of step
```

```
def get_current_price(coin):
```

```
    print("Getting current price...")
```

```
return 10000
```

```
price = get_current_price("bitcoin")
```

Moving Through the Stack

- pdb allows navigation between stack frames.
- Useful when functions call other functions.
- Helps inspect execution context.
- Supports moving up and down.
- Makes debugging nested calls easier.

up Command (u)

- Moves one frame higher.
- Similar to stepping out.
- Displays caller information.
- Useful for tracing errors.
- Default movement is one level.

```
(Pdb) up
```

```
(Pdb) u
```

down Command (d)

- Moves one frame lower.
- Returns to previously viewed frames.
- Helps inspect nested function calls.
- Useful after using up.
- Default movement is one level.

```
(Pdb) down
```

```
(Pdb) d
```

Stack Trace

- Shows the sequence of function calls.
- Helps locate error origins.
- Displays active frames.
- Reveals execution path.
- Critical for debugging exceptions.

where Command (w)

- Displays the current stack trace.
- Shows all active function calls.
- Marks the current frame.
- Indicates the current line number.
- Alias: bt.

(Pdb) where

(Pdb) w

Reading a Stack Trace

- Oldest frame appears first.
- Newest frame appears last.
- Current frame is marked with >.
- Current line is marked with ->.
- Helps identify execution flow.

Displaying Source Code

- pdb can display source code directly.
- Useful when debugging large files.
- Reduces context switching.
- Shows surrounding code.
- Supports line ranges.

list Command (l)

- Displays nearby source lines.
- Shows context around current execution point.
- Helps understand program flow.
- Can display specific ranges.
- Frequently used while debugging.

(Pdb) list

(Pdb) l

longlist Command (ll)

- Displays the entire current function.
- Provides more context than list.
- Useful for larger functions.
- Highlights current execution line.
- Simplifies navigation.

(Pdb) longlist

(Pdb) ll

Additional Debugging Tools

- Several tools complement pdb.
- Improve code quality and debugging efficiency.
- Detect errors before runtime.
- Automate formatting and analysis.
- Commonly used in professional development.

PyLint

- Static code analysis tool.
- Detects bugs and code smells.
- Enforces coding standards.

- Improves maintainability.
- Finds issues before execution.

`pylint app.py`

ipdb

- Enhanced interactive debugger.
- Built on top of pdb.
- Provides better user experience.
- Supports syntax highlighting.
- Easier to use than standard pdb.

`pip install ipdb`

Black

- Automatically formats Python code.
- Ensures consistent style.
- Reduces formatting debates.
- Improves readability.
- Works with any project.

`black app.py`

isort

- Automatically sorts imports.
- Keeps import sections organized.
- Improves code consistency.
- Integrates with Black.
- Simplifies maintenance.

`isort app.py`

mypy

- Static type checker for Python.
- Detects type-related issues.
- Improves code reliability.

- Supports type hints.
- Helps catch bugs early.